LAPORAN PRAKTIKUM ALGORITMA

DAN PEMROGRAMAN 2

MODUL 7 & 9

Sturct dan Array

Disusun oleh:

Ahmad Malik Arrayyan

109082500116

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhi

Renisa Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Tugas

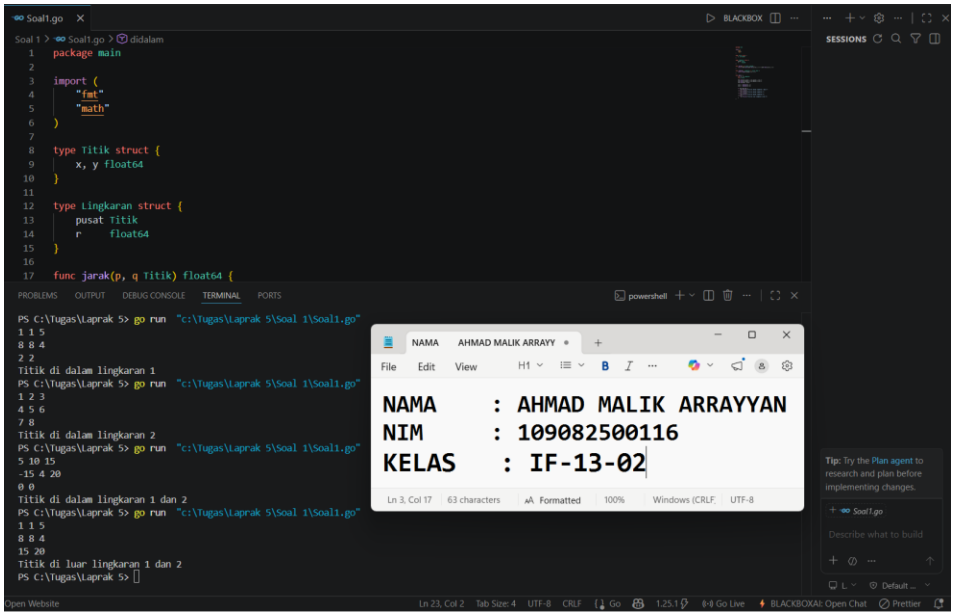
1. **Tugas 1**

   Source Code

```
package main

import (
    "fmt"
    "math"
)

type Titik struct {
    x, y float64
}

type Lingkaran struct {
    pusat Titik
    r     float64
}

func jarak(p, q Titik) float64 {
    return math.Sqrt(math.Pow(p.x-q.x, 2) + math.Pow(p.y-q.y,
2))
}

func didalam(c Lingkaran, p Titik) bool {
    return jarak(c.pusat, p) <= c.r
}

func main() {
    var l1, l2 Lingkaran
    var p Titik

    fmt.Scan(&l1.pusat.x, &l1.pusat.y, &l1.r)
    fmt.Scan(&l2.pusat.x, &l2.pusat.y, &l2.r)
```

```
        fmt.Scan(&p.x, &p.y)

        inL1 := didalam(l1, p)
        inL2 := didalam(l2, p)

        if inL1 && inL2 {
            fmt.Println("Titik di dalam lingkaran 1 dan 2")
        } else if inL1 {
            fmt.Println("Titik di dalam lingkaran 1")
        } else if inL2 {
            fmt.Println("Titik di dalam lingkaran 2")
        } else {
            fmt.Println("Titik di luar lingkaran 1 dan 2")
        }
    }
```

Screenshoot program



Deskripsi program

Program ini merupakan versi yang lebih canggih dari pendeteksi posisi titik karena sudah menerapkan konsep **Struct** atau tipe data terstruktur. Alih-alih mengelola variabel secara terpisah-pisah, saya membungkus data ke dalam objek yang logis: struct Titik untuk menampung koordinat x dan y, serta struct Lingkaran yang memiliki 'pusat' (bertipe Titik) dan jari-jari. Pendekatan ini membuat kode jauh lebih mudah dibaca dan dikelola karena mencerminkan cara kita berpikir di dunia nyata—bahwa sebuah lingkaran memang memiliki titik pusat dan ukuran.

Secara teknis, fungsi jarak sekarang menerima input berupa objek Titik, bukan lagi empat angka desimal yang membingungkan. Program menghitung jarak antara titik pusat lingkaran dengan titik target menggunakan rumus Pythagoras. Fungsi didalam kemudian memvalidasi apakah jarak tersebut masih berada dalam jangkauan jari-jari lingkaran. Di bagian main, penggunaan struct membuat proses input dan pengecekan kondisi if-else menjadi lebih ringkas. Dengan mengelompokkan data seperti ini, program tidak hanya berfungsi dengan akurat, tetapi juga memiliki struktur kode yang profesional dan siap untuk dikembangkan ke sistem koordinat yang lebih kompleks.

2. **Tugas 2**

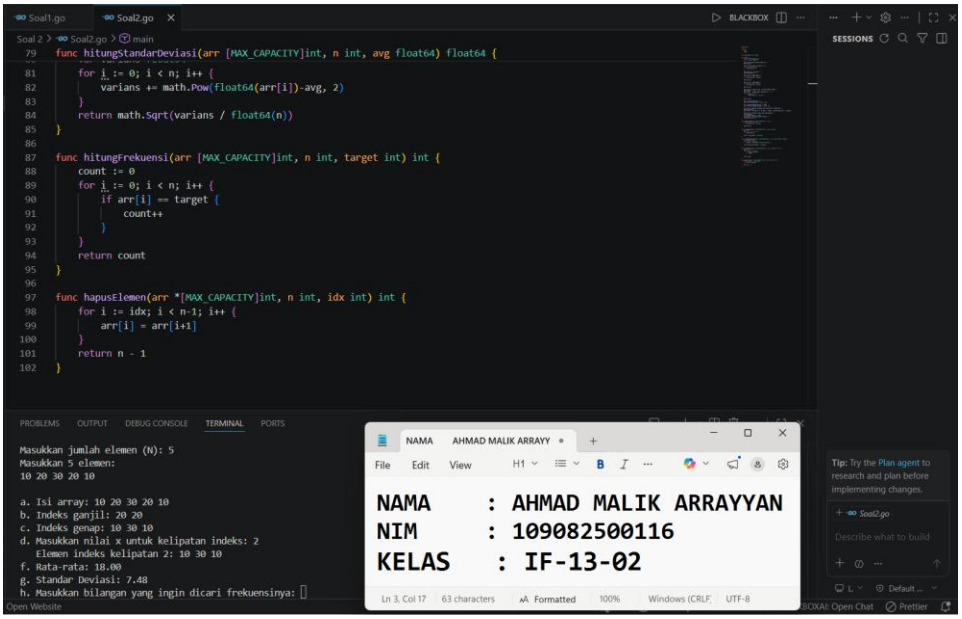
Source Code

```
package main
import (
    "fmt"
    "math"
)
const MAX_CAPACITY int = 100
func main() {
    var arr [MAX_CAPACITY]int
    var n, x, idx, target int
    fmt.Print("Masukkan jumlah elemen (N): ")
    fmt.Scan(&n)
    fmt.Printf("Masukkan %d elemen:\n", n)
    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }
    fmt.Print("\na. Isi array: ")
    tampilkanSemua(arr, n)
    fmt.Print("b. Indeks ganjil: ")
    for i := 1; i < n; i += 2 {
        fmt.Printf("%d ", arr[i])
    }
    fmt.Println()
    fmt.Print("c. Indeks genap: ")
    for i := 0; i < n; i += 2 {
        fmt.Printf("%d ", arr[i])
    }
    fmt.Println()
    fmt.Print("d. Masukkan nilai x untuk kelipatan
indeks: ")
    fmt.Scan(&x)
    fmt.Printf("   Elemen indeks kelipatan %d: ", x)
```

```
    for i := 0; i < n; i++ {
        if i%x == 0 {
            fmt.Printf("%d ", arr[i])
        }
    }
    fmt.Println()
    avg := hitungRataRata(arr, n)
    fmt.Printf("f. Rata-rata: %.2f\n", avg)
    sd := hitungStandarDeviasi(arr, n, avg)
    fmt.Printf("g. Standar Deviasi: %.2f\n", sd)
    fmt.Print("h. Masukkan bilangan yang ingin dicari
frekuensinya: ")
    fmt.Scan(&target)
    fmt.Printf("   Frekuensi %d: %d kali\n", target,
hitungFrekuensi(arr, n, target))
    fmt.Print("e. Masukkan indeks yang ingin dihapus: ")
    fmt.Scan(&idx)
    n = hapusElemen(&arr, n, idx)
    fmt.Print("   Isi array setelah dihapus: ")
    tampilkanSemua(arr, n)
}
func tampilkanSemua(arr [MAX_CAPACITY]int, n int) {
    for i := 0; i < n; i++ {
        fmt.Printf("%d ", arr[i])
    }
    fmt.Println()
}
func hitungRataRata(arr [MAX_CAPACITY]int, n int)
float64 {
    sum := 0
    for i := 0; i < n; i++ {
        sum += arr[i]
    }
```

```
    return float64(sum) / float64(n)
}
func hitungStandarDeviasi(arr [MAX_CAPACITY]int, n int,
avg float64) float64 {
    var varians float64
    for i := 0; i < n; i++ {
        varians += math.Pow(float64(arr[i])-avg, 2)
    }
    return math.Sqrt(varians / float64(n))
}
func hitungFrekuensi(arr [MAX_CAPACITY]int, n int,
target int) int {
    count := 0
    for i := 0; i < n; i++ {
        if arr[i] == target {
            count++
        }
    }
    return count
}
func hapusElemen(arr *[MAX_CAPACITY]int, n int, idx int)
int {
    for i := idx; i < n-1; i++ {
        arr[i] = arr[i+1]
    }
    return n - 1
}
```

Screenshoot program



Deskripsi program

Program ini merupakan implementasi lengkap dari pengolahan data menggunakan **Array Statis**. Di sini, saya menggunakan sebuah array dengan kapasitas tetap sebesar 100 elemen sebagai wadah utama. Program ini tidak hanya sekadar menyimpan data, tetapi juga memiliki berbagai fitur analisis dan manipulasi data yang modular. Saya membagi tugas-tugas kompleks ke dalam beberapa fungsi khusus, seperti menghitung rata-rata, mencari frekuensi kemunculan angka tertentu, hingga menghitung standar deviasi untuk melihat persebaran data menggunakan pustaka matematika math.

Salah satu bagian yang paling krusial adalah fungsi hapusElemen. Di sini saya menggunakan **Pointer** untuk mengakses array aslinya secara langsung. Logika penghapusan yang saya gunakan bukan benar-benar menghapus memori, melainkan melakukan 'pergeseran elemen' (shifting); elemen di sebelah kanan indeks yang dihapus ditarik satu langkah ke kiri untuk mengisi kekosongan, dan ukuran logis array (n) dikurangi satu.

Selain itu, program ini juga mendemonstrasikan fleksibilitas akses indeks, mulai dari menampilkan angka pada posisi ganjil, genap, hingga kelipatan tertentu. Dengan struktur seperti ini, program mampu menyajikan analisis data statistik dasar sekaligus melakukan perubahan struktur data secara dinamis dalam wadah yang statis.

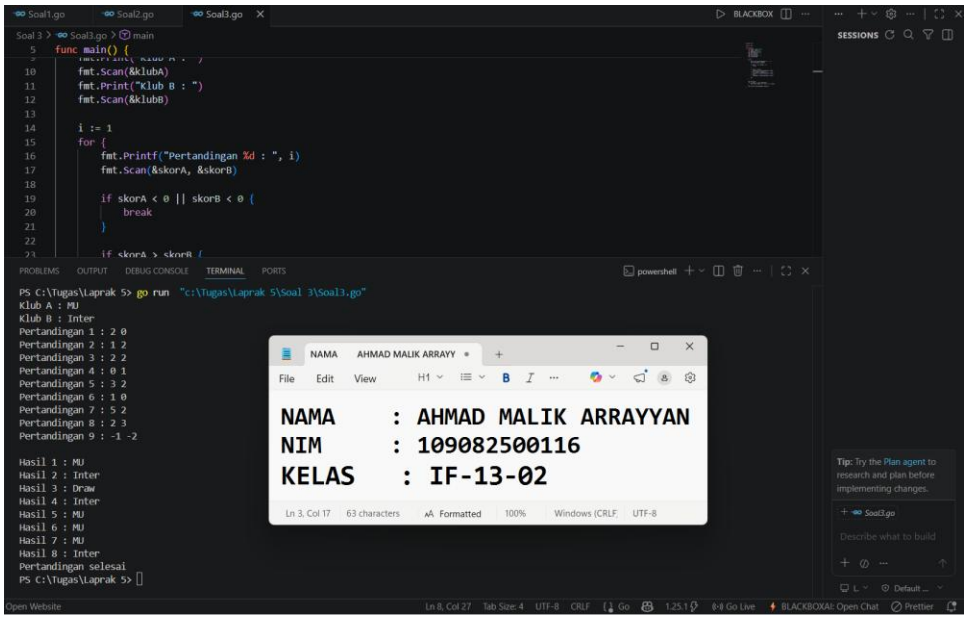
3. **Tugas 3**

Source Code

```
package main

import "fmt"

func main() {
    var klubA, klubB string
    var skorA, skorB int
    var pemenang []string
    fmt.Print("Klub A : ")
    fmt.Scan(&klubA)
    fmt.Print("Klub B : ")
    fmt.Scan(&klubB)

    i := 1
    for {
        fmt.Printf("Pertandingan %d : ", i)
        fmt.Scan(&skorA, &skorB)

        if skorA < 0 || skorB < 0 {
            break
        }

        if skorA > skorB {
            pemenang = append(pemenang, klubA)
        } else if skorB > skorA {
            pemenang = append(pemenang, klubB)
        } else {
            pemenang = append(pemenang, "Draw")
        }
        i++
```

```
        }


        fmt.Println()
        for j, hasil := range pemenang {
            fmt.Printf("Hasil %d : %s\n", j+1, hasil)
        }
        fmt.Println("Pertandingan selesai")
    }
```

Screenshoot program



Deskripsi program

Program ini dirancang sebagai sistem pencatat hasil pertandingan bola yang fleksibel, di mana jumlah pertandingannya tidak ditentukan di awal. Logika utamanya menggunakan **Slice**, yaitu versi dinamis dari array. Berbeda dengan array statis yang ukurannya kaku, slice pemenang di sini berfungsi seperti daftar yang bisa terus memanjang otomatis menggunakan fungsi append setiap kali ada hasil pertandingan baru yang masuk.

Alur kerjanya dimulai dengan menetapkan nama dua klub yang bertanding. Kemudian, program masuk ke dalam perulangan tanpa henti (*infinite loop*). Di dalam perulangan ini, sistem akan terus meminta input skor. Jika pengguna memasukkan skor negatif (misalnya -1), itulah sinyal bagi program untuk berhenti menggunakan perintah break.

Setiap kali skor dimasukkan, program menjalankan logika pencabangan: jika skor A lebih besar, nama Klub A disimpan; jika B lebih besar, nama Klub B disimpan; dan jika sama, maka status 'Draw' yang dicatat. Setelah perulangan berhenti, program

menggunakan perulangan range untuk menyisir kembali seluruh daftar yang tersimpan di dalam slice dan menampilkannya sebagai laporan hasil akhir pertandingan secara urut.

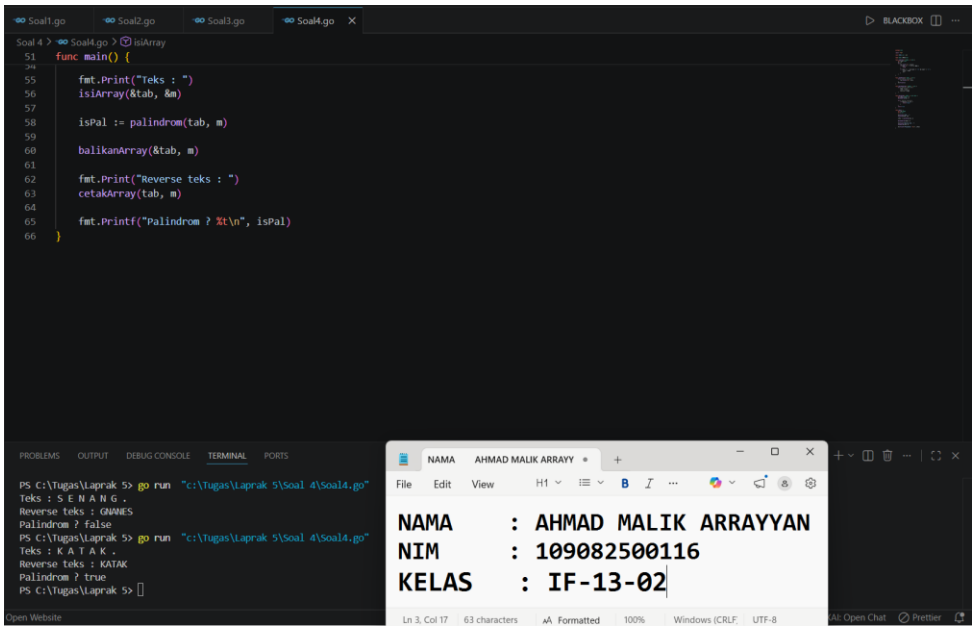
4. **Tugas 4**

Source code

```
package main


import "fmt"


const NMAX int = 127


type tabel [NMAX]rune


func isiArray(t *tabel, n *int) {
    var input rune
    *n = 0
    for {
        fmt.Scanf("%c", &input)
        if input == '.' || *n >= NMAX {
            break
        }
        if input != ' ' && input != '\n' && input != '\r' {
            t[*n] = input
            *n++
        }
    }
}


func cetakArray(t tabel, n int) {
    for i := 0; i < n; i++ {
        fmt.Printf("%c", t[i])
    }
    fmt.Println()
}


func balikanArray(t *tabel, n int) {
    for i := 0; i < n/2; i++ {
```

```
        temp := t[i]
        t[i] = t[n-1-i]
        t[n-1-i] = temp
    }
}

func palindrom(t tabel, n int) bool {
    var tAsli tabel = t
    balikanArray(&t, n)

    for i := 0; i < n; i++ {
        if tAsli[i] != t[i] {
            return false
        }
    }
    return true
}

func main() {
    var tab tabel
    var m int

    fmt.Print("Teks : ")
    isiArray(&tab, &m)

    isPal := palindrom(tab, m)

    balikanArray(&tab, m)

    fmt.Print("Reverse teks : ")
    cetakArray(tab, m)

    fmt.Printf("Palindrom ? %t\n", isPal)
}
```

Screenshoot program



Deskripsi program

Program ini dirancang untuk memproses sekumpulan karakter guna menentukan apakah teks tersebut merupakan sebuah **Palindrom**—yaitu teks yang jika dibaca dari depan maupun belakang hasilnya tetap sama. Logika program ini dibangun secara modular menggunakan tipe data rune, yang sangat cocok dalam bahasa Go untuk menangani karakter tunggal.

Prosesnya dimulai dengan fungsi isiArray yang membaca input karakter demi karakter hingga menemukan tanda titik (.) sebagai penanda berhenti. Menariknya, fungsi ini secara cerdas mengabaikan spasi atau baris baru agar pengecekan Palindrom menjadi lebih akurat. Kunci utama pengecekannya ada pada fungsi balikanArray yang menggunakan algoritma **Two-Pointer Swap**; program hanya perlu berjalan setengah dari panjang array (n/2) untuk menukar posisi karakter paling depan dengan paling belakang secara simetris.

Untuk menentukan status Palindrom, fungsi palindrom membuat salinan array asli, membalik salah satunya, lalu membandingkan keduanya karakter demi karakter. Jika semua posisi karakter cocok setelah dibalik, maka fungsi mengembalikan nilai true. Terakhir, program akan menampilkan teks yang sudah dibalik tersebut beserta status kebenaran Palindromnya, memberikan analisis yang lengkap terhadap input teks yang diberikan.